

Eleanor Schema Documentation

Table of contents

Overview	2
Tables	2
orders	3
variable_space	4
scratch	5
kernel	6
elements	7
species	8
Reactants	8
aqueous_reactants	9
element_reactants	10
fixed_gas_reactants	10
gas_reactants	11
mineral_reactants	12
solid_solution_reactants	12
solid_solution_reactant_end_members	13
special_reactants	14
special_reactant_compositions	14
suppressions	15
suppression_exceptions	16
equilibrium_space	17
equilibrium_reactants	22
equilibrium_elements	23
equilibrium_aqueous_species	24
equilibrium_gases	25
equilibrium_pure_solids	26
equilibrium_solid_solutions	27
equilibrium_end_members	29
equilibrium_redox_reactions	30
Appendix - Example Order	31

Overview

The primary function of **Eleanor** is to accept a problem detailing how to select fully-specified aqueous speciation problems, referred to as an “[order](#)”, equilibrate them, and curate the resulting data.

Each such selected problem corresponds to a point (row) in the “variable space” ([variable_space](#) table). Each order may then have any number of associated variable space rows. The properties of the variable space points are broken out over a number of tables, e.g. the [elements](#) table contains the elemental composition of the fluid before speciation, the [mineral_reactants](#) table contains the amount and titration rate of each mineral to be reacted with the system, etc...

Speciation is then handled by the kernel specified in the order, which may produce zero or more points (rows) in the “equilibrium space” ([equilibrium_space](#) table). Each row for a given variable space point represents some form or degree of equilibration. For example, the [eq36](#) kernel performs the equilibration in two stages, first speciating the fluid alone (the “eq3” stage) and then equilibrating the fluid with the reactants (the “eq6” stage). As such, any variable space point equilibrated with the [eq36](#) kernel will have at least two equilibrium points associated with it (more on this below). As with variable space points, several tables contain detailed information about the equilibrium space points, e.g. the [equilibrium_elements](#) table contains the elemental composition of the fluid after the given speciation, the [equilibrium_reactants](#) table contains information about how much of each reactant has been, and how much remains to be reacted, etc...

Tables

This document describes each table in the schema and its relationship to the other tables. For brevity we use a few abbreviations as follows:

Key	
PK	Primary Key
FK	Foreign Key
N	Nullable

A table carries a primary key (**PK**)—always named `id`, always a `UUID`—only when something else refers to it. The leaf tables, such as [elements](#) and [equilibrium_aqueous_species](#), are reached only through their parent and so have no `id` of their own.

Ids are UUIDv7 values **Eleanor** generates, not integers the database assigns. Being version 7, they embed a millisecond timestamp in their leading bits, so ordering by `id` orders by creation time.

As described in [Overview](#), the **Eleanor** database is somewhat hierarchical with the [orders](#) table as the root, zero or more entries in the [variable_space](#) table referring to an entry in [orders](#), and with zero or more entries in the [equilibrium_space](#) table referring to an entry in the [variable_space](#) table. These relationships are reflected in two ways in this document:

1. The document headings are structured such that each table refers to the table one-level above it.
2. The foreign keys (**FK**) are noted in the tables, and the associated column name links to the referenced table. These column names are generally of the form `*_id`. The main exceptions are one-to-one tables where `id` is both the primary key and the foreign key, e.g. [kernel](#) and [scratch](#), which each reuse [variable_space](#) ids.

Most columns in the **Eleanor** database are `NOT NULL`, meaning they are guaranteed to have a value, though that value may be some standard value representing “absent” or “missing”, e.g. `-inf`. Columns that are nullable (**N**) are denoted as such.

The examples throughout this document truncate ids to their first group, as `cf165798-...`, so the tables fit the page. The same truncation stands for the same row everywhere, so a foreign key can be matched to the row it references by eye.

orders

PK	FK	N	Column Name	Type	Description	Unit
✓			id	UUID	The order identifier	
			name	VARCHAR	The name of the order	
			tags	TEXT[]	Tags for organizing orders (default: '{}')	
			eleanor_version	VARCHAR	The version of eleanor used to run the order	
			raw	JSONB	The raw, JSON-encoded contents of the order	
			create_date	DATETIME	The datetime at which the order was created	

When the user runs an order, **Eleanor** stores an entry in the `orders` table which includes the generated `id`, the name of the order (extracted from the YAML/TOML/JSON file), optional `tags` for organization, and the version of **Eleanor** used to execute the order.

Indexes:

- INDEX `orders_name_idx` on (`name`)
- INDEX `orders_tags_idx` on (`tags`) USING GIN

The `raw` column stores the original content of the order file in JSON format. It does not carry an `id`: the identifier belongs to this table's `id` column, generated when the row is inserted, and an order file may not declare one.

The `create_date` value is the datetime at which the order was initially created.

Example:

```
SELECT id, name, tags, eleanor_version, create_date FROM orders;
```

```
+-----+-----+-----+-----+-----+
| id      | name                | tags  | eleanor_version | create_date          |
+-----+-----+-----+-----+-----+
| e49c9daa-... | A Wild Example Order | {test} | 0.19.0          | 2026-01-09 10:59:01.98 |
+-----+-----+-----+-----+-----+
```

variable_space

PK	FK	N	Column Name	Type	Description	Unit
✓			<code>id</code>	UUID	The variable space point identifier	
	✓		<code>order_id</code>	UUID	The order id for which the point was generated	
			<code>water_mass</code>	DOUBLE	The mass of water in the system	<i>kg</i>
			<code>temperature</code>	DOUBLE	The temperature	<i>°C</i>
			<code>pressure</code>	DOUBLE	The pressure	<i>bar</i>
			<code>exit_code</code>	INTEGER	The exit code generated by the kernel (non-zero for failure)	
	✓		<code>error</code>	TEXT	The error message generated by the kernel on failure, if any	
			<code>create_date</code>	DATETIME	The datetime when the variable space point was created	
			<code>start_date</code>	DATETIME	The datetime when the equilibration started	
			<code>complete_date</code>	DATETIME	The datetime when the equilibration completed	

Indexes:

- INDEX `variable_space_order_id_exit_code_idx` on (`order_id`, `exit_code`)
- INDEX `variable_space_exit_code_failed_idx` on (`exit_code`) WHERE `exit_code` <> 0
- INDEX `variable_space_temperature_idx` on (`temperature`)
- INDEX `variable_space_pressure_idx` on (`pressure`)

When the user runs **Eleanor**, a specified number of variable space points are selected based on the constraints provided by that order. Each point represents the unspeciatted initial state of a system and includes such information as:

- Water Mass
- Temperature
- Pressure
- Kernel Settings ([kernel](#))
- Elemental Composition ([elements](#))
- Species Constraints ([species](#))
- [Reactant Specifications](#), including:
 - Aqueous Reactants ([aqueous_reactants](#))
 - Element Reactants ([element_reactants](#))
 - Fixed Gas Reactants ([fixed_gas_reactants](#))
 - Gas Reactants ([gas_reactants](#))
 - Mineral Reactants ([mineral_reactants](#))
 - Solid Solution Reactants ([solid_solution_reactants](#))
 - Special Reactants ([special_reactants](#))
- Suppressions ([suppressions](#))

The `variable_space` table stores the core information about the variable space point per row. This includes a generated `id`, the `water_mass` (mass of water in the system in kilograms; defaults to 1.0 if unspecified in the order), the `temperature` and `pressure` of the system, and the datetime at which it was generated. Each point refers back to the order to which it belongs via the `order_id`.

After generating the variable space point, **Eleanor** proceeds to speciate the system using the user-specified kernel. The `exit_code` generated by the kernel as well as the datetimes of when the speciation started and completed are stored in the `variable_space` table as well. If the kernel fails (non-zero `exit_code`), the `error` column stores the associated error message.

Example:

```
SELECT * FROM variable_space ORDER BY id LIMIT 1;
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
| id      | order_id | water_mass | temperature | pressure | exit_code | error | create_date           | start_date           |
complete_date         |
+-----+-----+-----+-----+-----+-----+-----+-----+
| cf165798-... | e49c9daa-... | 1.0      | 300.0      | 500.0    | 0        | <null> | 2026-01-09 10:59:02.541519 | 2026-01-09 10:59:02.546294
| 2026-01-09 10:59:02.843071 |
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
```

scratch

PK	FK	N	Column Name	Type	Description	Unit
✓	✓		<code>id</code>	UUID	The scratch entry identifier corresponds to the variable space id	
			<code>zip</code>	BYTEA	A zip archive of the input files used and output files generated when equilibrating the variable space point. This includes a traceback when an error occurs	

The input files used and output files generated by the kernel when speciating a `“variable space”` point are stored in a `zip` archive in the `scratch` table. The `scratch` row will have the same `id` as the associated variable space point.

By default, **Eleanor** only stores scratch entries if an error occurs while speciating the variable space point, as signified by a non-zero exit code from the kernel. In such a case a `traceback.txt` file is included in the `zip` archive detailing the error. The user may specify, at runtime, that scratch be collected for all variable space speciations. In that case, no `traceback.txt` file is included in the `zip` archive.

Example:

```
SELECT * FROM scratch WHERE id = 'cf165798-...';
```

```
+-----+-----+
+-----+-----+
```

```

| id          | zip      |
|-----+-----|
| cf165798-... | <binary> |
+-----+-----+

```

kernel

PK	FK	N	Column Name	Type	Description	Unit
✓	✓		<code>id</code>	UUID	The kernel entry identifier corresponds to the variable space point	
			<code>type</code>	VARCHAR	The type of the kernel used, e.g. <code>eq36</code>	
			<code>settings</code>	JSONB	A JSON-encoded object of the settings passed to the kernel	

Eleanor is designed to support user-provided kernels which may require specific configuration or settings. Those settings are stored in the `kernel` table, one row for each [variable space](#) point. The kernel settings row will have the same `id` as the variable space point to which it belongs. The `type` of the kernel is stored alongside a JSON encoded `settings` object, the form of which is kernel-specific.

While, at the moment, only the `eleanor.kernel.eq36` kernel is provided out-of-the-box, future versions may include other kernel implementations or the user may design and implement their own. It is then difficult to design a schema for storing the settings. Using a JSON object makes things simpler given PostgreSQL's native JSON support.

Example:

```
SELECT * FROM kernel WHERE id = 'cf165798-...';
```

```

+-----+-----+-----+
| id          | type | settings                                     |
|-----+-----+-----|
| cf165798-... | eq36 | {"model": 0, "timeout": null, "basis_map": {}, ... } |
+-----+-----+-----+

```

elements

PK	FK	N	Column Name	Type	Description	Unit
	✓		<code>variable_space_id</code>	UUID	The id of the variable space point for which elemental molality was generated	
			<code>name</code>	VARCHAR	The name of the element (atomic symbol)	
			<code>log_molality</code>	DOUBLE	The log molality of the element in the variable space solution	log <i>molal</i>

- INDEX `elements_variable_space_id_name_idx` on (`variable_space_id`, `name`)

The elemental composition of the system's fluid must be provided as part of the specification of a variable space point. Each element in the fluid is stored in the `elements` table with the name of the element and the `log_molality` that was chosen when the variable space was generated. Each element references the variable space point it is associated with by the `variable_space_id`.

Example:

```
SELECT *
FROM elements
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+
| variable_space_id | name | log_molality |
+-----+-----+-----+
| cf165798-...     | C   | -9.0         |
| cf165798-...     | Ca  | -9.0         |
| cf165798-...     | Cl  | -9.0         |
| cf165798-...     | Fe  | -9.0         |
| cf165798-...     | Mg  | -9.0         |
| cf165798-...     | N   | -9.0         |
| cf165798-...     | Na  | -9.0         |
| cf165798-...     | Si  | -9.0         |
+-----+-----+-----+
```

species

PK	FK	N	Column Name	Type	Description	Unit
	✓		variable_space_id	UUID	The id of the variable space point for which the species molality/activity/fugacity was generated	
			name	VARCHAR	The name of the species	
			value	DOUBLE	The log molality/activity/fugacity of the species in the variable space solution	variable

- INDEX `species_variable_space_id_name_idx` on (`variable_space_id`, `name`)

Constraints on the aqueous and gaseous species in the system may, though in some cases must, be provided as part of the specification of a variable space point, such as the pH and fO_2 of the system. Each species constraint is stored in the `species` table with the name of the species and the `value` of the constraint (log molality, log activity or log fugacity as appropriate) that was chosen when the variable space was generated. Each element references the variable space point it is associated with by the `variable_space_id`.

Example:

```
SELECT *
FROM species
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+
| variable_space_id | name  | value |
+-----+-----+-----+
| cf165798-...     | H+    | -7.0  |
| cf165798-...     | O2(g) | -1.0  |
+-----+-----+-----+
```

Reactants

Several types of reactant can be specified in an order, facilitating the simulation of a wide variety of systems. These types are:

- Aqueous Reactants ([aqueous_reactants](#))
- Element Reactants ([element_reactants](#))
- Fixed Gas Reactants ([fixed_gas_reactants](#))
- Gas Reactants ([gas_reactants](#))
- Mineral Reactants ([mineral_reactants](#))
- Solid Solution Reactants ([solid_solution_reactants](#))
- Special Reactants ([special_reactants](#))

and each the details of reactants are stored in tables by type. However, they mostly follow the same basic form:

1. Each table has a `variable_space_id` that associates the entry with the [variable space](#) point,
2. The `name` of the reactant,

3. The molar amount of the reactant, `log_moles`, and
4. Either a `titration_rate` or `log_fugacity` as appropriate for the reactant type.

Only `solid_solution_reactants` and `special_reactants` also carry an `id`, because each has a child table—`solid_solution_reactant_end_members` and `special_reactant_compositions`—that refers back to it.

aqueous_reactants

PK	FK	N	Column Name	Type	Description	Unit
	✓		<code>variable_space_id</code>	UUID	The id of the variable space point for which the aqueous reactant amount and titration rate were generated	
			<code>name</code>	VARCHAR	The name of the aqueous reactant	
			<code>log_moles</code>	DOUBLE	The log amount of the reactant to be reacted with the fluid	$\log mol$
			<code>titration_rate</code>	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX `aqueous_reactants_variable_space_id_name_idx` on (`variable_space_id`, `name`)

An aqueous reactant models the addition (or removal) of a fixed molar quantity (`amount`) of an aqueous species into the system at a ξ -rate (`titration_rate`).

Example:

```
SELECT *
FROM aqueous_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+
| variable_space_id | name  | log_moles | titration_rate |
+-----+-----+-----+-----+
| cf165798-...     | CaCl2 | -2.0      | 1.0             |
+-----+-----+-----+-----+
```

element_reactants

PK	FK	N	Column Name	Type	Description	Unit
	✓		variable_space_id	UUID	The id of the variable space point for which the element reactant amount and titration rate were generated	
			name	VARCHAR	The name of the element reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	log <i>mol</i>
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	<i>mol</i> / ξ

- INDEX `element_reactants_variable_space_id_name_idx` on (`variable_space_id`, `name`)

An element reactant is just a special case of [special_reactant](#) in which a fixed molar quantity (`amount`) of a single (unionized) element is added to the bulk composition of the fluid at some ξ -rate (`titration_rate`).

Example:

```
SELECT *
FROM element_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+
| variable_space_id | name | log_moles | titration_rate |
+-----+-----+-----+-----+
| cf165798-...      | Si   | -2.0      | 1.0            |
+-----+-----+-----+-----+
```

fixed_gas_reactants

PK	FK	N	Column Name	Type	Description	Unit
	✓		variable_space_id	UUID	The id of the variable space point for which the fixed gas reactant amount and fugacity were generated	
			name	VARCHAR	The name of the fixed gas reactant	
			log_moles	DOUBLE	The log amount of the fixed gas buffer	log <i>mol</i>
			log_fugacity	DOUBLE	The log fugacity of will be titrated into the fluid	<i>mol</i> / ξ

- INDEX `fixed_gas_reactants_variable_space_id_name_idx` on (`variable_space_id`, `name`)

A fixed gas reactant models gas exchange with a large, external gas reservoir at a fixed fugacity. Rather than the gas being “titrated” in, the `log_fugacity` is enforced through mass exchange with a buffer with initial size `amount`.

Example:

```
SELECT *
FROM fixed_gas_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+
| variable_space_id | name   | log_moles | log_fugacity |
|-----+-----+-----+-----|
| cf165798-...     | CO2(g) | -0.3      | -3.5         |
+-----+-----+-----+-----+
```

gas_reactants

PK	FK	N	Column Name	Type	Description	Unit
	✓		<u>variable_space_id</u>	UUID	The id of the variable space point for which the gas reactant amount and titration rate were generated	
			name	VARCHAR	The name of the gas reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	log mol
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ ξ

- INDEX gas_reactants_variable_space_id_name_idx on (variable_space_id, name)

A gas reactant models reacting a fixed molar quantity (amount) of a gaseous species with the fluid at some ξ -rate (titration_rate).

Example:

```
SELECT *
FROM gas_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+
| variable_space_id | name   | log_moles | titration_rate |
|-----+-----+-----+-----|
| cf165798-...     | N2(g)  | -0.3      | 1.0           |
+-----+-----+-----+-----+
```

mineral_reactants

PK	FK	N	Column Name	Type	Description	Unit
	✓		variable_space_id	UUID	The id of the variable space point for which the mineral reactant amount and titration rate were generated	
			name	VARCHAR	The name of the mineral reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	log <i>mol</i>
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	<i>mol</i> / ξ

- INDEX `mineral_reactants_variable_space_id_name_idx` on (`variable_space_id`, `name`)

A mineral reactant models reacting a fixed molar quantity (`amount`) of a pure mineral with the fluid at some ξ -rate (`titration_rate`).

Example:

```
SELECT *
FROM mineral_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+
| variable_space_id | name      | log_moles | titration_rate |
+-----+-----+-----+-----+
| cf165798-...      | hematite  | -0.3      | 1.0            |
+-----+-----+-----+-----+
```

solid_solution_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	UUID	The solid solution identifier, specific to the variable space point	
	✓		variable_space_id	UUID	The id of the variable space point for which the solid solution amount and titration rate were generated	
			name	VARCHAR	The name of the solid solution	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	log <i>mol</i>
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	<i>mol</i> / ξ

- INDEX `solid_solution_reactants_variable_space_id_name_idx` on (`variable_space_id`, `name`)

A solid solution reactant models reacting a fixed molar quantity (`amount`) of a solid solution with the fluid at some ξ -rate (`titration_rate`). Each such solid solution must include the mole-fraction of each of its end members. The end member data is stored separately in the [solid_solution_reactant_end_members](#) table.

Example:

```
SELECT *
FROM solid_solution_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+-----+
| id      | variable_space_id | name      | log_moles | titration_rate |
+-----+-----+-----+-----+-----+
| b3b05895-... | cf165798-...      | olivine-ss | -0.3      | 1.0            |
+-----+-----+-----+-----+-----+
```

solid_solution_reactant_end_members

PK	FK	N	Column Name	Type	Description	Unit
	✓		solid_solution_reactant_id	UUID	The id of the solid solution for which this is an end member	
			name	VARCHAR	The name of the end member	
			fraction	DOUBLE	The mole-fraction of the end member in the solid solution	unitless

- CONSTRAINT fraction_in_range CHECK (0.0 <= fraction AND fraction <= 1.0)
- INDEX solid_solution_reactant_end_members_ssr_id_name_idx on (solid_solution_reactant_id, name)

Each [solid solution reactant](#) has a fixed distribution of end members, represented by the mole-fraction of the total solid solution.

Example:

```
SELECT em.*
FROM solid_solution_reactant_end_members AS em
JOIN solid_solution_reactants AS ssr ON ssr.id = em.solid_solution_reactant_id AND variable_space_id = 'cf165798-...'
ORDER BY em.name;
```

```
+-----+-----+-----+
| solid_solution_reactant_id | name      | fraction |
+-----+-----+-----+
| b3b05895-...              | fayalite  | 0.6      |
| b3b05895-...              | forsterite | 0.4      |
+-----+-----+-----+
```

special_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	UUID	The special reactant identifier, specific to the variable space point	
	✓		variable_space_id	UUID	The id of the variable space point for which the special reactant amount and titration rate were generated	
			name	VARCHAR	The name of the special reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	log mol
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ ξ

- INDEX `special_reactants_variable_space_id_name_idx` on (variable_space_id, name)

A special reactant models reacting a fixed molar quantity (`amount`) of a user-specified molecular composition with the fluid at some ξ -rate (`titration_rate`). Each special reactant must have include its stoichiometric composition, which is stored separately in the [special_reactant_compositions](#) table.

Example:

```
SELECT *
FROM special_reactants
WHERE variable_space_id = 'cf165798-...'
ORDER BY name;
```

```
+-----+-----+-----+-----+-----+
| id      | variable_space_id | name           | log_moles | titration_rate |
+-----+-----+-----+-----+-----+
| 8b31573d-... | cf165798-...      | CalciumHydroxide | -2.0      | 1.0            |
+-----+-----+-----+-----+-----+
```

special_reactant_compositions

PK	FK	N	Column Name	Type	Description	Unit
✓			special_reactant_id	UUID	The id of the special reactant	
			element	VARCHAR	The name of the element in the special reactant	
			count	INTEGER	The stoichiometric count of the element in the special reactant	

- INDEX `special_reactant_compositions_special_reactant_id_element_idx` on (special_reactant_id, element)

Each [special reactant](#) has a fixed stoichiometric composition. Each `element` and its occurrence (`count`) in the reactant is stored in a row of the `special_reactant_composition` table.

Example:

```
SELECT c.*
FROM special_reactant_compositions AS c
JOIN special_reactants AS sr ON sr.id = c.special_reactant_id AND sr.variable_space_id = 'cf165798-...'
ORDER BY c.element;
```

```
+-----+-----+-----+
| special_reactant_id | element | count |
+-----+-----+-----+
| 8b31573d-...       | Ca      | 1      |
| 8b31573d-...       | H       | 2      |
| 8b31573d-...       | O       | 2      |
+-----+-----+-----+
```

suppressions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	UUID	The suppression identifier	
	✓		variable_space_id	UUID	The id of the variable space point	
		✓	name	VARCHAR	The name of the species/phase to suppress, e.g. H_2O_2 , methane, olivine	
		✓	type	VARCHAR	The type of species/phase to suppress, e.g. “solid solutions”	

- CONSTRAINT suppressions_well_defined CHECK (name is not null or type is not null)
- INDEX suppressions_variable_space_id_idx on (variable_space_id)

Users can specify, within an order, that some species or entire phase type should be suppressed from forming. When a phase type is specified, exceptions can be provided. Those exceptions are stored separately in the [suppression_exceptions](#) table.

Example:

```
SELECT * FROM suppressions WHERE variable_space_id = 'cf165798-...';
```

```
+-----+-----+-----+-----+
| id          | variable_space_id | name      | type    |
+-----+-----+-----+-----+
| ca8e88ce-... | cf165798-...      | METHANE   | <null>  |
| d5c329aa-... | cf165798-...      | antigorite | <null>  |
| f914672f-... | cf165798-...      | <null>     | mineral |
+-----+-----+-----+-----+
```

suppression_exceptions

PK	FK	N	Column Name	Type	Description	Unit
			name	VARCHAR	The name the species/phase to except, e.g. H2O2, methane, olivine	
	✓		suppression_id	UUID	The id of the suppression for which to except	

- INDEX [suppression_exceptions_suppression_id_idx](#) on ([suppression_id](#))

Each exception is stored as a row in the [suppression_exceptions](#) table, referring back to the suppression by the [suppression_id](#) value.

Example:

```
SELECT se.*
FROM suppression_exceptions AS se
JOIN suppressions AS s ON s.id = se.suppression_id AND s.variable_space_id = 'cf165798-...'
ORDER BY se.name;
```

```
+-----+-----+
| name      | suppression_id |
+-----+-----+
| magnetite | f914672f-...   |
+-----+-----+
```

equilibrium_space

PK	FK	N	Column Name	Type	Description	Unit
✓			id	UUID	The equilibrium space point identifier	
	✓	✓	variable_space_id	UUID	The id of the variable space point for which the equilibrium was found	
			stage	VARCHAR	The stage of the equilibration, e.g. "eq3", "eq6"	
		✓	log_xi	DOUBLE	The logarithm of the ξ -step at which this equilibrium was found	unitless
			temperature	DOUBLE	The temperature	°C
			pressure	DOUBLE	The pressure	bar
			"pH"	DOUBLE	The pH on the NBS scale	unitless
			"log_f02"	DOUBLE	The logarithm of the oxygen (O_2) fugacity	log bar
			"Eh"	DOUBLE	The redox potential	V
			log_activity_water	DOUBLE	The logarithm of the activity of water	unitless
			log_ionic_strength	DOUBLE	The logarithm of the ionic strength of the solution	log molal
			solute_mass	DOUBLE	The total mass of all dissolved species	g
			solvent_mass	DOUBLE	The mass of the solvent (water)	g
			solution_mass	DOUBLE	The mass of the solution	g
			tds	DOUBLE	The total dissolved solute (TDS); dissolved solute mass per kilogram of solution	mg/kg
	✓		reactant_mass_reacted	DOUBLE	The total mass reacted into the system up to this ξ step	g
	✓		reactant_mass_remaining	DOUBLE	The total mass remaining to be reacted	g
			start_date	DATETIME	The datetime when the equilibration started	
			complete_date	DATETIME	The datetime when the equilibration completed	
			custom_properties	JSONB	Less common solution properties and any non-standard equilibrium properties computed by the kernel, stored as a JSON object	

Indexes:

- INDEX equilibrium_space_variable_space_id_stage_idx on (variable_space_id, stage)
- INDEX equilibrium_space_stage_variable_space_id_idx on (stage, variable_space_id)
- INDEX equilibrium_space_temperature_idx on (temperature)
- INDEX equilibrium_space_pressure_idx on (pressure)

When **Eleanor** speciates a fluid, it is the kernel's responsibility to perform the calculations and provide one or more equilibrium space points. Each of these points, consisting of identifying information (id, variable_space_id, stage and log_xi) and a wide range of system properties (e.g. temperature, pressure, "pH", etc...), is stored as a row in the

`equilibrium_space` table. Additional, information about the system, such as the quantity of reactants reacted and remaining (`equilibrium_reactants`), the elemental composition of the fluid (`equilibrium_elements`), the molecular composition of the fluid (`equilibrium_aqueous_species`), etc..., are stored in auxiliary tables that link back to the equilibrium space point via a foreign key reference on their `equilibrium_space_id` column.

For example, the builtin `eleanor.kernel.eq36.Kernel` will produce at least 2 equilibrium space points: one representing the speciation of the fluid alone (the "eq3" stage), and the final equilibrium after reacting the fluid with any reactants (the "eq6" stage). In the case that the user configured the kernel with the `track_paths: true` option, (many) more than 2 equilibrium space points will be produced: the "eq3" point, and some number of intermediate points between the "eq3" system and the final "eq6" representing a titration path. That is, each point along the titration path represents the equilibration of the fluid with an increasing quantity of reactant. The progress along the path is tracked by ξ (stored as `log_xi`). The `log_xi` value will be `NULL` for "eq3" stage points as ξ is undefined.

Different kernels, and different stages processed by a given kernel, may result in some columns being either `NULL` or `-inf` for one reason or another. This can cause confusion. Take for example the case of `log_xi` before. The "eq3" stage points will have `log_xi = NULL`, and the first point of a titration path produced in the "eq6" stage will have `log_xi = -inf` ($\xi = 0$ at the start of the path). However, if the user did not enable path tracking or set the kernel's print settings judiciously, then the first "eq6" stage point may have a finite `log_xi`. Other examples include:

1. Since mass is not transferred in the "eq3" stage, columns such as `reactant_mass_reacted` and `reactant_mass_remaining` will be `NULL`.

Less commonly queried solution properties (such as `mole_fraction_water`, `log_gamma_water`, `pe`, `Ah`, charge-balance quantities, solid mass and volume changes, etc.) are stored in the `custom_properties` column. Each kernel may also compute and store additional non-standard properties there.

Example (Without Paths):

```
SELECT
  id,
  variable_space_id,
  stage,
  log_xi,
  temperature,
  pressure,
  "pH",
  start_date,
  complete_date
FROM equilibrium_space
WHERE variable_space_id = 'cf165798-...'
ORDER BY stage, log_xi;
```

id	variable_space_id	stage	log_xi	temperature	pressure	pH	start_date	complete_date
322ba26a-...	cf165798-...	eq3	<null>	300.0	500.0	5.4541	2026-01-09 10:59:02.54	2026-01-09 10:59:02.55
c38a9eea-...	cf165798-...	eq6	-0.3000002024454176	300.0	500.0	5.3627	2026-01-09 10:59:02.55	2026-01-09 10:59:02.80

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+

Example (With Paths):

```

SELECT
  id,
  variable_space_id,
  stage,
  log_xi,
  temperature,
  pressure,
  "pH",
  start_date,
  complete_date
FROM equilibrium_space
WHERE variable_space_id = 'cf165798-...'
ORDER BY stage, log_xi;

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
+-----+									
id	variable_space_id	stage	log_xi	temperature	pressure	pH	start_date	complete_date	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
322ba26a-...	cf165798-...	eq3	<null>	300.0	500.0	5.4541	2026-01-09 11:02:59.49	2026-01-09 11:02:59.50	
c38a9eea-...	cf165798-...	eq6	-inf	300.0	500.0	5.4536	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
86ba526a-...	cf165798-...	eq6	-8.0	300.0	500.0	5.454	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
81f617fa-...	cf165798-...	eq6	-7.0	300.0	500.0	5.4574	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
fe7c3f0e-...	cf165798-...	eq6	-6.443549140627881	300.0	500.0	5.4706	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
090f64a9-...	cf165798-...	eq6	-6.0	300.0	500.0	5.5368	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
2ec28757-...	cf165798-...	eq6	-5.0	300.0	500.0	6.0213	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
37cd5c7b-...	cf165798-...	eq6	-4.734955647654595	300.0	500.0	6.1948	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
93720f8c-...	cf165798-...	eq6	-4.0	300.0	500.0	6.5286	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
3c939e9a-...	cf165798-...	eq6	-3.8915466083537007	300.0	500.0	6.6473	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
b78f5bf9-...	cf165798-...	eq6	-3.22983825514633	300.0	500.0	6.3563	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
2cc13b93-...	cf165798-...	eq6	-3.0	300.0	500.0	6.3854	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
0f840ce2-...	cf165798-...	eq6	-2.0	300.0	500.0	6.2304	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
edb77b57-...	cf165798-...	eq6	-2.0	300.0	500.0	6.2304	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
e10a638c-...	cf165798-...	eq6	-1.8955811793524053	300.0	500.0	5.6519	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
466d301e-...	cf165798-...	eq6	-1.4778975981455702	300.0	500.0	5.7453	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
76237ce3-...	cf165798-...	eq6	-1.4727156101843601	300.0	500.0	5.6938	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
b6a3323e-...	cf165798-...	eq6	-1.3782070009986855	300.0	500.0	5.3294	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
868b4459-...	cf165798-...	eq6	-1.0	300.0	500.0	5.3303	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	
ef5c2458-...	cf165798-...	eq6	-0.3000002024454176	300.0	500.0	5.3627	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80	

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+

equilibrium_reactants

PK	FK	N	Column Name	Type	Description	Unit
	✓		<code>equilibrium_space_id</code>	UUID	The id of the equilibrium space point	
			<code>name</code>	VARCHAR	The name of the reactant	
			<code>affinity</code>	DOUBLE	The affinity of the reactant	
			<code>relative_rate</code>	DOUBLE	The molar rate of reactant titration relative to the total molar quantity of all reactants	mol/mol
			<code>log_moles_reacted</code>	DOUBLE	The moles of reactant that have been reacted up to this ξ step	$\log mol$
			<code>log_moles_remaining</code>	DOUBLE	The moles of reactant that remain to be reacted	$\log mol$
			<code>log_mass_reacted</code>	DOUBLE	The mass of the reactant that have been reacted up to this ξ step	$\log kg$
			<code>log_mass_remaining</code>	DOUBLE	The mass of the reactant that remain to be reacted	$\log kg$

If an `equilibrium_space` entry represents a stage in which mass is transferred, (i.e. reacted or precipitated) and the system has N reactants, then the point will be associated with N rows in the `equilibrium_reactants` table, one for each reactant.

Note: The exception to this rule is for `fixed_gas_reactants`, which the `eleanor.kernel.eq36` kernel treats differently. Fixed gas reactants are enforced by requiring equilibrium with a fictitious solid phase in `equilibrium_pure_solids`, e.g. $CO_2(g)$ as a fixed gas reactant will produce a pure solid of "`fix_fCO2(g)`", and do not appear in the `equilibrium_reactants` table.

The `equilibrium_reactants` table is most useful for two purposes:

1. **Quality assurance.** At the end of the equilibration, all the `log_moles_remaining` ought to be `-inf` (no reactant). If that is not the case, then the reaction did not progress far enough and kernel settings likely need to be tweaked.
2. **Path Tracking.** If titration path tracking is enabled, the `log_moles_reacted` should monotonically increase while `log_moles_remaining` monotonically decreases along the path (ordered by `log_xi` from the `equilibrium_space` table).

Example:

SELECT

```
equilibrium_space_id AS eid,
name,
affinity,
relative_rate AS rate,
log_moles_reacted AS moles_reacted,
log_moles_remaining,
log_mass_reacted,
log_mass_remaining
FROM equilibrium_reactants
WHERE equilibrium_space_id = 'c38a9eea-...'
ORDER BY name;
```

eid	name	affinity	rate	moles_reacted	log_moles_remaining	log_mass_reacted	log_mass_remaining
c38a9eea-...	CaCl2	9999999.0	0.0	-2.0	-inf	0.045283851395135605	-inf
c38a9eea-...	CalciumHydroxide	9999999.0	0.0	-2.0	-inf	-0.13021109763031236	-inf
c38a9eea-...	hematite	1.43	1.0	-0.29999760285774607	-inf	1.903285375549671	-inf
c38a9eea-...	N2(g)	9999999.0	1.0	-0.29999760285774607	-inf	1.1473671077937864	-inf
c38a9eea-...	olivine-ss	1.7744	1.0	-0.29999760285774607	-inf	1.9517453891382763	-inf
c38a9eea-...	Si	9999999.0	0.0	-2.0	-inf	-0.5515101084889142	-inf

equilibrium_elements

PK	FK	N	Column Name	Type	Description	Unit
✓			equilibrium_space_id	UUID	The id of the equilibrium space point	
			name	VARCHAR	The name of the element (atomic symbol)	
			log_molality	DOUBLE	The molality of the element in solution	log <i>molal</i>
			mass_fraction	DOUBLE	The fraction of solution mass accounted for by the element	unitless

Every [equilibrium_space](#) entry will be associated with some number of rows in the [equilibrium_elements](#) table, one for each element in the fluid. These rows record the [name](#) and [log_molality](#) of the element in the fluid and the [mass_fraction](#) of the fluid that element represents.

If no precipitates form, then these rows should agree with the entries in the [elements](#) table. However, that is rarely the case, so expect them to differ.

Example:

```
SELECT *
FROM equilibrium_elements
WHERE equilibrium_space_id = 'c38a9eea-...'
ORDER BY name;
```

equilibrium_space_id	name	log_molality	mass_fraction
c38a9eea-...	C	-5.5935330597128265	3.01624e-08
c38a9eea-...	Ca	-2.1571530866467845	0.000274913
c38a9eea-...	Cl	-1.6971649242940714	0.000701302
c38a9eea-...	Fe	-6.133895871377041	4.04134e-08
c38a9eea-...	H	2.0457140589408676	0.110299
c38a9eea-...	Mg	-4.097879571162284	1.910899999999998e-06
c38a9eea-...	N	0.002835353178722796	0.0138865
c38a9eea-...	Na	-8.998192757497696	2.273839999999998e-11
c38a9eea-...	O	1.7443829627474854	0.8747969999999999
c38a9eea-...	Si	-2.838188943951825	4.01526e-05

equilibrium_aqueous_species

PK	FK	N	Column Name	Type	Description	Unit
	✓		equilibrium_space_id	UUID	The id of the equilibrium space point	
			name	VARCHAR	The name of the aqueous species, e.g. "H2O2"	
			log_molality	DOUBLE	The logarithm of the molality of that species in solution	log <i>molal</i>
			log_activity	DOUBLE	The logarithm of the activity of that species in solution	unitless
			log_gamma	DOUBLE	The logarithm of the activity coefficient of that species in solution	unitless

- INDEX equilibrium_aqueous_species_name_log_molality_idx on (name, log_molality)
- INDEX equilibrium_aqueous_species_equilibrium_space_id_name_idx on (equilibrium_space_id, name)

Every [equilibrium_space](#) entry will be associated with some number of rows in the [equilibrium_aqueous_species](#) table, one for each species in the fluid. These rows record the name, log_molality, log_activity and log_gamma (log-activity coefficient) of the species in accordance with the activity model used by the kernel.

Example:

```
SELECT *
FROM equilibrium_aqueous_species
WHERE equilibrium_space_id = 'c38a9eea-...'
ORDER BY log_molality DESC, name
LIMIT 10;
```

equilibrium_space_id	name	log_molality	log_activity	log_gamma
c38a9eea-...	N2	-0.3092	-0.3092	0.0
c38a9eea-...	NH3	-1.7195	-1.7195	0.0
c38a9eea-...	Cl-	-1.7863	-1.9078	-0.1215
c38a9eea-...	NH4+	-2.2149	-2.3465	-0.1316
c38a9eea-...	CaCl+	-2.4406	-2.5666	-0.126
c38a9eea-...	Ca+2	-2.4947	-2.9493	-0.4545
c38a9eea-...	SiO2	-2.8385	-2.8385	0.0
c38a9eea-...	H2	-3.1232	-3.1179	0.0052
c38a9eea-...	CaOH+	-4.0071	-4.1331	-0.126
c38a9eea-...	Mg+2	-4.3002	-4.7213	-0.4211

equilibrium_gases

PK	FK	N	Column Name	Type	Description	Unit
	✓		equilibrium_space_id	UUID	The id of the equilibrium space point	
			name	VARCHAR	The name of the gas, e.g. "H2(g)"	
			log_fugacity	DOUBLE	The logarithm of the fugacity of the gas at this ξ step	log bar

Every [equilibrium_space](#) entry will be associated with some number of rows in the [equilibrium_gases](#) table, one for each gas phase. These rows record the name and log_fugacity.

Example:

```
SELECT *
FROM equilibrium_gases
WHERE equilibrium_space_id = 'c38a9eea-...'
ORDER BY log_fugacity DESC, name;
```

```
+-----+-----+-----+
| equilibrium_space_id | name  | log_fugacity |
+-----+-----+-----+
| c38a9eea-...        | N2(g) | 2.41361      |
| c38a9eea-...        | H2O(g) | 1.85299      |
| c38a9eea-...        | H2(g)  | -0.58253     |
| c38a9eea-...        | NH3(g) | -0.83984     |
| c38a9eea-...        | CO2(g) | -3.5         |
| c38a9eea-...        | CH4(g) | -4.06719     |
| c38a9eea-...        | CO(g)  | -7.60332     |
| c38a9eea-...        | NO(g)  | -23.50603    |
| c38a9eea-...        | N2O(g) | -26.10458    |
| c38a9eea-...        | O2(g)  | -34.26937    |
+-----+-----+-----+
```

equilibrium_pure_solid

PK	FK	N	Column Name	Type	Description	Unit
	✓		equilibrium_space_id	UUID	The id of the equilibrium space point	
			name	VARCHAR	The name of the pure solid, e.g. “fayalite”	
			log_qk	DOUBLE	The saturation index of the pure solid phase	unitless
			affinity	DOUBLE	the affinity of the pure solid	kcal
			log_moles	DOUBLE	The logarithm of the net moles of the solid that have precipitated up to this ξ step	log mol
			log_mass	DOUBLE	The logarithm of the net mass of the solid that has precipitated up to this ξ step	log g
			log_volume	DOUBLE	The logarithm of the volume of the solid that has precipitated	log cm ³

- INDEX equilibrium_pure_solid_name_present_idx on (name, equilibrium_space_id) WHERE log_moles > '-Infinity'::double precision
- INDEX equilibrium_pure_solid_equilibrium_space_id_name_idx on (equilibrium_space_id, name) WHERE log_moles > '-Infinity'::double precision

Every [equilibrium_space](#) entry will be associated with some number of rows in the [equilibrium_pure_solid](#) table, one for each pure solid (not a solid solution). These rows record the name, saturation index (log_qk), affinity, log_moles, log_mass and log_volume of the solid phase.

Notes:

1. The solids that appear in this table include both solids that form as well as those that *could* have formed based on the composition of the system. For example, the solids associated with the "eq3" stage will all be hypothetical as the kernel does not partition mass in that stage. As such, the value in the `log_moles`, `log_mass` and `log_volume` columns will be `-inf`. Similarly, those columns will be `-inf` for "eq6"-stage equilibrium space points if the phase does not actually precipitate.
2. In many cases, volume data is lacking. Even if a solid precipitates, it will *likely* have a $V = 0$ (`log_volume = -inf`).

Example:

```
SELECT *
FROM equilibrium_pure_solids
WHERE equilibrium_space_id = 'c38a9eea-...'
ORDER BY log_mass DESC, log_qk DESC, name
LIMIT 10;
```

equilibrium_space_id	name	log_qk	affinity	log_moles	log_mass	log_volume
c38a9eea-...	magnetite	-0.0	-0.0	-0.2891	2.0755104645244136	-inf
c38a9eea-...	antigorite	1.67537	4.39389	-inf	-inf	-inf
c38a9eea-...	tremolite	-0.0	-0.0	-inf	-inf	-inf
c38a9eea-...	talc	-0.12292	-0.32237	-inf	-inf	-inf
c38a9eea-...	chrysotile	-0.30052	-0.78816	-inf	-inf	-inf
c38a9eea-...	diopside	-0.42799	-1.12247	-inf	-inf	-inf
c38a9eea-...	hematite	-0.54526	-1.43002	-inf	-inf	-inf
c38a9eea-...	enstatite	-0.739	-1.93813	-inf	-inf	-inf
c38a9eea-...	quartz	-0.8933	-2.34282	-inf	-inf	-inf
c38a9eea-...	goethite	-0.90359	-2.36978	-inf	-inf	-inf

equilibrium_solid_solutions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	UUID	The equilibrium solid solution identifier	
	✓		equilibrium_space_id	UUID	The id of the equilibrium space point	
			name	VARCHAR	The name of the solid solution, e.g. "olivine"	
			log_qk	DOUBLE	The saturation index of the solid solution	unitless
			affinity	DOUBLE	The affinity of the solid solution	kcal
			log_moles	DOUBLE	The logarithm of the net moles of the solid solution that have precipitated up to this ξ step	log mol
			log_mass	DOUBLE	The logarithm of the net mass of the solid solution that has precipitated up to this ξ step	log g

PK	FK	N	Column Name	Type	Description	Unit
			log_volume	DOUBLE	The logarithm of the volume of the solid solution that has precipitated	log cm ³

- INDEX equilibrium_solid_solutions_name_present_idx on (name, equilibrium_space_id) WHERE log_moles > '-Infinity'::double precision
- INDEX equilibrium_solid_solutions_equilibrium_space_id_name_idx on (equilibrium_space_id, name) WHERE log_moles > '-Infinity'::double precision

Every [equilibrium_space](#) entry will be associated with some number of rows in the [equilibrium_solid_solutions](#) table, one for each solid solution. These rows record the name, saturation index (log_qk), affinity, log_moles, log_mass and log_volume of the solid solution. Additionally, the end member composition of the solid solutions are stored in the [equilibrium_end_members](#) auxiliary table.

Notes:

1. The solid solutions that appear in this table include both solid solutions that form as well as those that *could* have formed based on the composition of the system. For example, the solid solutions associated with the "eq3" stage will all be hypothetical as the kernel does not partition mass in that stage. As such, the value in the log_moles, log_mass and log_volume columns will be -inf. Similarly, those columns will be -inf for "eq6"-stage equilibrium space points if the phase does not actually precipitate.
2. In many cases, volume data is lacking. Even if a solid precipitates, it will *likely* have a $V = 0$ (log_volume = -inf).

Example:

```
SELECT *
FROM equilibrium_solid_solutions
WHERE equilibrium_space_id = 'c38a9eea-...'
ORDER BY log_mass DESC, log_qk DESC, name
LIMIT 10;
```

id	equilibrium_space_id	name	log_qk	affinity	log_moles	log_mass	log_volume
f08e76bc-...	c38a9eea-...	talcc-ss	-0.0	-0.0	-1.0435	1.5450472044011843	-inf
6bb83eee-...	c38a9eea-...	serpentine	-0.0	-0.0	-1.2932	1.1838390370564211	-inf
466705d0-...	c38a9eea-...	amphib_ternary-ss	-0.0	-0.0	-2.1849	0.7248327204665614	-inf
c4bb1837-...	c38a9eea-...	ca-amphib_binary-ss	-0.0	-0.0	-inf	-inf	-inf
01e2ac2d-...	c38a9eea-...	clinopyroxene-ss	-0.41699	-1.09361	-inf	-inf	-inf
ad4c543e-...	c38a9eea-...	orthopyroxene-ss	-0.59034	-1.54825	-inf	-inf	-inf
ce9fe19a-...	c38a9eea-...	ideal_olivine-ss	-0.64149	-1.6824	-inf	-inf	-inf
fe1bd033-...	c38a9eea-...	olivine-ss	-0.64149	-1.6824	-99999.0	-inf	-inf
ef1084b4-...	c38a9eea-...	brucite-ss	-1.2999	-3.40916	-inf	-inf	-inf
5250ee1e-...	c38a9eea-...	carbonate-ss	-4.07608	-10.69009	-inf	-inf	-inf

equilibrium_end_members

PK	FK	N	Column Name	Type	Description	Unit
✓			<code>equilibrium_solid_solution_id</code>	UUID	The id of the equilibrium solid solution	
			<code>name</code>	VARCHAR	The name of the end member, e.g. "fayalite"	
			<code>log_qk</code>	DOUBLE	The saturation index of the end member	unitless
			<code>affinity</code>	DOUBLE	The affinity of the end member	kcal
			<code>log_moles</code>	DOUBLE	The logarithm of the net moles of the end member that have precipitated up to this ξ step	log mol
			<code>log_mass</code>	DOUBLE	The logarithm of the net mass of the end member that has precipitated up to this ξ step	log g
			<code>log_volume</code>	DOUBLE	The logarithm of the volume of the end member that has precipitated	log cm ³

- INDEX `equilibrium_end_members_name_present_idx` on (name, equilibrium_solid_solution_id) WHERE log_moles > '-Infinity'::double precision
- INDEX `equilibrium_end_members_equilibrium_solid_solution_id_name_idx` on (equilibrium_solid_solution_id, name) WHERE log_moles > '-Infinity'::double precision

Every `equilibrium_solid_solutions` entry will be associated with some number of rows in the `equilibrium_end_members` table, one for each end member. These rows record the name, saturation index (`log_qk`), affinity, `log_moles`, `log_mass` and `log_volume` of the solid phase.

Notes:

1. The end members that appear in this table include both end members that form as well as those that *could* have formed based on the composition of the system. For example, end members of solid solutions associated with the "eq3" stage will all be hypothetical as the kernel does not partition mass in that stage. As such, the value in the `log_moles`, `log_mass` and `log_volume` columns will be `-inf`. Similarly, those columns will be `-inf` for "eq6"-stage equilibrium space points if the phase does not actually precipitate.
2. In many cases, volume data is lacking. Even if a solid precipitates, it will *likely* have a $V = 0$ (`log_volume = -inf`).

Example:

```
SELECT *
FROM equilibrium_end_members
WHERE equilibrium_solid_solution_id = 'f08e76bc-...'
ORDER BY log_mass DESC, log_qk DESC, name;
```

```
+-----+-----+-----+-----+-----+-----+-----+
| equilibrium_solid_solution_id | name          | log_qk | affinity | log_moles | log_mass          | log_volume |
+-----+-----+-----+-----+-----+-----+-----+
| f08e76bc-...                  | talc          | -0.0   | -0.0     | -1.0845   | 1.4944328987263986 | -inf       |
| f08e76bc-...                  | minnesotaite  | -0.0   | -0.0     | -2.0892   | 0.5865197925102743 | -inf       |
+-----+-----+-----+-----+-----+-----+-----+
```

equilibrium_redox_reactions

PK	FK	N	Column Name	Type	Description	Unit
	✓		equilibrium_space_id	UUID	The id of the equilibrium space point	
			couple	VARCHAR	The name of the redox couple	
			"Eh"	DOUBLE	The redox potential	V
			pe	DOUBLE	The negative logarithm of the hypothetical electron activity	unitless
			"log_f02"	DOUBLE	The logarithm of the oxygen (O ₂) fugacity	log bar
			"Ah"	DOUBLE	The redox affinity	kcal

Each `equilibrium_space` entry can be associated with one or more rows in the `equilibrium_redox_reactions` table, one for each redox constraint on the system.

Note: In the present version of **Eleanor**, there will typically a single `equilibrium_redox_reactions` entry for each `equilibrium_space` entry, and the values in this table will be wholly redundant with `equilibrium_space`.

Example:

```
SELECT redox.*
FROM equilibrium_redox_reactions AS redox
JOIN equilibrium_space AS es ON es.id = redox.equilibrium_space_id AND es.variable_space_id = 'cf165798-...';
```

equilibrium_space_id	couple	Eh	pe	log_f02	Ah
322ba26a-...	DEFAULT	0.373	3.2836	-1.0	8.612
c38a9eea-...	DEFAULT	-0.562	-4.9403	-34.269	-12.957

Appendix - Example Order

```
name: A Wild Example Order
creator: 39 Alpha Research, Team One
date: 2026-01-09
notes: This is a nonsensical order intended to demonstrate a wide
```

```
kernel:
  kind: eq36
  model: b-dot
  charge_balance: H+
  track_paths: true
  eq6_config:
    xi_max: 1
    print_step_interval: 1
```

```
elements:
  C: -5
  Mg: -9
  Fe: -9
  Na: [-4, -5]
  Si:
    min: -9
    max: -6
  Ca:
    mean: -4
    stddev: 0.01
    min: -5
    max: -3
```

```
species:
  H+: -7
  O2(g): -1
```

```
reactants:
  CaCl2:
    type: aqueous
    amount: -3.0
    titration_rate: 1.0
  Na:
    type: element
    amount: -4.0
    titration_rate: 1.0
```

```
CO2(g):
  type: fixed gas
  amount: -0.3
  log_fugacity: -3.5
N2(g):
  type: gas
  amount: -0.3
  titration_rate: 1.0
hematite:
  type: mineral
  amount: -0.3
  titration_rate: 1.0
olivine-ss:
  type: solid solution
  amount: -0.3
  titration_rate: 1.0
  end_members:
    fayalite: 0.6
    forsterite: 0.4
CalciumHydroxide:
  type: special
  amount: -0.3
  titration_rate: 1.0
  composition:
    Ca: 1
    O: 2
    H: 2
```

```
Suppressions:
- METHANE
- antigorite
- type: mineral
except:
- magnetite
```